

SEMANA 3 ACADEMIA MONTERREY

Despliegue de una aplicación Spring Boot en AWS EC2

En esta actividad se realizó el despliegue de una aplicación desarrollada con Spring Boot en un servidor virtual de Amazon Web Services (AWS), utilizando el servicio Amazon EC2.

El objetivo principal fue conocer el proceso básico necesario para llevar una aplicación Java desde un entorno local hasta un servidor en la nube, incluyendo la configuración de acceso, conexión remota mediante SSH, preparación del entorno, generación del archivo ejecutable `.jar`, transferencia del artefacto al servidor y ejecución de la aplicación.

El proceso realizado fue el siguiente:

1. Creación y configuración de una cuenta/usuario IAM.
 2. Creación de las credenciales necesarias para trabajar con AWS.
 3. Creación de una instancia EC2.
 4. Generación y descarga de una llave privada `.pem`.
 5. Configuración del acceso mediante SSH.
 6. Conexión al servidor EC2 desde Git Bash.
 7. Verificación de los recursos disponibles en el servidor.
 8. Instalación de Java 21.
 9. Compilación y empaquetado de la aplicación Maven en el equipo local.
 10. Transferencia del archivo `.jar` hacia EC2 mediante SCP.
 11. Verificación de la integridad del archivo transferido.
 12. Ejecución de la aplicación Spring Boot en el servidor.
 13. Acceso a Swagger desde un navegador mediante la IP pública de la instancia.
 14. Prueba de autenticación y creación de una tarea mediante la API.
-

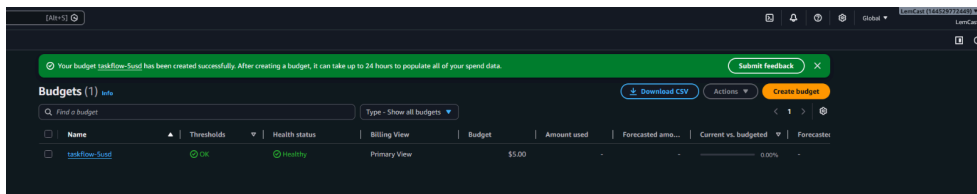
2. Configuración inicial de AWS

2.1 Creación del budget y el usuario IAM

Como primer paso se configuró el acceso a AWS mediante un usuario de IAM (Identity and Access Management).

IAM permite administrar usuarios, credenciales y permisos dentro de AWS. En lugar de utilizar directamente la cuenta principal de AWS para las actividades, se creó un usuario destinado a trabajar con los recursos necesarios para el ejercicio.

Durante la configuración se asignaron los permisos necesarios para poder administrar los recursos utilizados durante la práctica. Antes de iniciar, se estableció trabajar con un presupuesto máximo de 5.00 USD, el cual se realizó con la herramienta de budget de AWS.



2.2 Creación de la llave de acceso para EC2

Para poder conectarse posteriormente al servidor EC2 mediante SSH se creó una llave de acceso.

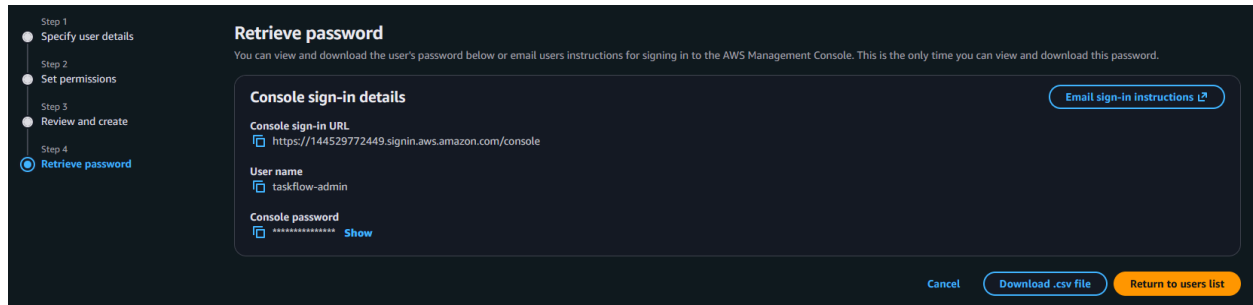
AWS permite generar un par de llaves compuesto por:

- Una llave pública, que queda asociada a la instancia.
- Una llave privada, descargada por el usuario como archivo `.pem`.

En este ejercicio la llave privada se guardó localmente con el nombre:

`taskflow-key.pem`

Esta llave es necesaria para autenticar la conexión SSH al servidor. Por seguridad, la llave privada debe mantenerse protegida y no debe compartirse públicamente.



3. Creación de la instancia EC2

Posteriormente se creó una instancia de Amazon EC2 para funcionar como servidor de la aplicación.

EC2 permite disponer de una máquina virtual en la infraestructura de AWS sin necesidad de contar físicamente con un servidor propio.

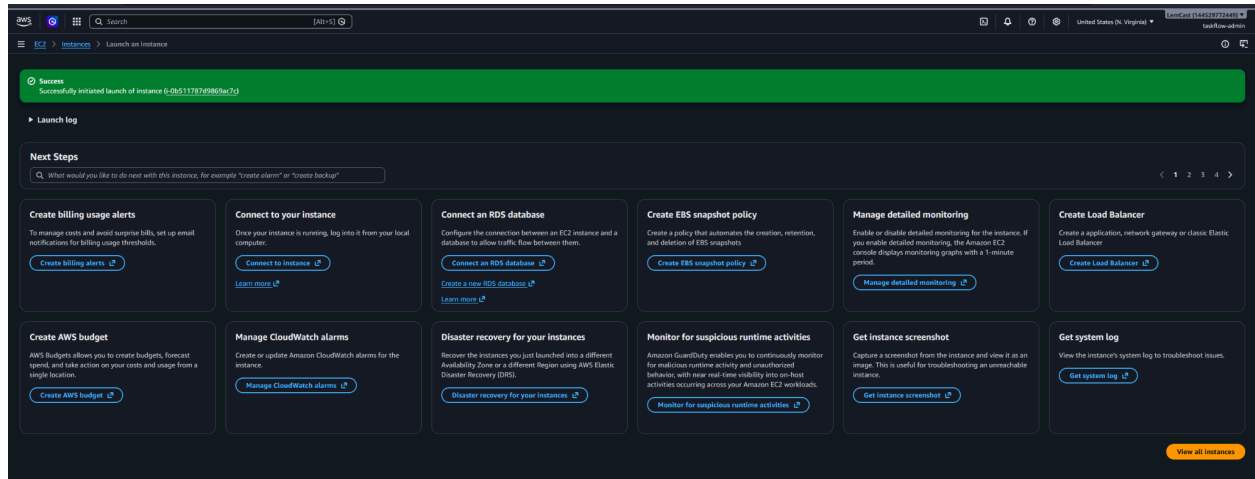
Para la práctica se utilizó una instancia basada en Amazon Linux 2023. La instancia recibió una dirección IP pública que posteriormente se utilizó para conectarse mediante SSH y para acceder a la aplicación desde un navegador.

También se configuró el grupo de seguridad asociado a la instancia para permitir las conexiones necesarias.

Entre los puertos utilizados se encuentran:

- **22/TCP:** utilizado para la conexión SSH.
- **8080/TCP:** utilizado posteriormente por la aplicación Spring Boot.

La IP pública establecida fué 54.87.12.162



4. Conexión al servidor mediante SSH

Una vez creada la instancia EC2, se utilizó Git Bash en el equipo local para establecer una conexión SSH con el servidor.

La llave privada se encontraba dentro de una carpeta local denominada **key**.

Primero se establecieron los permisos adecuados para la llave:

```
chmod 400 taskflow-key.pem
```

Posteriormente se realizó la conexión mediante:

```
ssh -i taskflow-key.pem ec2-user@54.87.12.162
```

Donde:

- **ssh** inicia una conexión mediante el protocolo Secure Shell.
- **-i** indica la llave privada que será utilizada para autenticarse.
- **taskflow-key.pem** corresponde a la llave privada.
- **ec2-user** es el usuario utilizado para acceder a Amazon Linux.
- **54.87.12.162** corresponde a la dirección pública asignada a la instancia EC2.

expresada en megabytes.

Después se verificó la instalación mediante:

```
java -version
```

El resultado confirmó que Java 21 estaba disponible en la instancia.

La instalación de Java permite que el servidor pueda ejecutar posteriormente el archivo `.jar` generado por el proyecto.

```
[ec2-user@ip-172-31-21-223 ~]$ java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-21-223 ~]$ |
```

7. Compilación de la aplicación Maven

Debido a las limitaciones de recursos de la instancia EC2, la aplicación no se compiló directamente en el servidor.

El proyecto se encontraba localmente dentro de la carpeta:

```
academyMty/taskflow-api
```

Desde esta ubicación se ejecutó:

```
mvn -q -DskipTests package
```

Este comando utiliza Maven para construir el proyecto y generar el artefacto ejecutable de la aplicación.

La opción:

```
-DskipTests
```

indica que durante este proceso no se ejecutarán las pruebas automatizadas.

Aaron Ulises Garza Rangel

Después de completar el proceso de compilación se verificó la existencia del archivo `.jar` generado:

```
ls -lh target/*.jar
```

El archivo generado se encontraba dentro de la carpeta:

```
target/
```

Este archivo representa el artefacto que posteriormente sería enviado al servidor EC2.

8. Transferencia del archivo JAR a EC2

Una vez generado el archivo `.jar`, se utilizó SCP (Secure Copy Protocol) para transferirlo desde el equipo local hacia la instancia EC2.

La transferencia se realizó mediante:

```
scp -i ../../key/taskflow-key.pem target/taskflow-api-*.jar ec2-user@<IP-PÚBLICA>:~/taskflow-api.jar
```

En este caso, la ruta:

```
../../key/taskflow-key.pem
```

se utilizó debido a la estructura de carpetas del proyecto.

El archivo fue enviado al directorio principal del usuario `ec2-user` dentro del servidor y se renombró como:

```
taskflow-api.jar
```

De esta manera, el flujo utilizado fue:

Equipo local

|

| Maven

▼

taskflow-api.jar

|

| SCP

▼

Servidor EC2

|

▼

~/taskflow-api.jar

El proyecto completo no fue enviado ni compilado en EC2. Únicamente se transfirió el artefacto ejecutable generado localmente.

9. Verificación de la integridad del archivo

Después de transferir el archivo al servidor, se verificó que el archivo recibido fuera exactamente el mismo que el generado localmente.

En el equipo local se obtuvo el hash SHA-256 mediante:

```
shasum -a 256 target/taskflow-api-*.jar
```

Posteriormente, dentro de EC2 se calculó el hash del archivo recibido:

```
sha256sum ~/taskflow-api.jar
```

Los valores obtenidos fueron comparados.

Si ambos hashes coinciden, significa que el archivo transferido no sufrió modificaciones durante el proceso de transferencia.

Este procedimiento permite comprobar la integridad del artefacto antes de ejecutarlo en el servidor.

[illegible]

10. Ejecución de la aplicación en EC2

Una vez que el archivo `.jar` se encontraba correctamente en el servidor, se inició la aplicación mediante:

```
nohup java -jar taskflow-api.jar > app.log 2>&1 &
```

El comando realiza varias acciones:

- `java -jar taskflow-api.jar` ejecuta la aplicación Spring Boot.
- `nohup` permite que el proceso continúe ejecutándose aunque la sesión SSH se cierre.
- `> app.log` redirige la salida de la aplicación hacia el archivo `app.log`.
- `2>&1` redirige también los mensajes de error hacia el mismo archivo.
- `&` ejecuta el proceso en segundo plano.

Para observar los mensajes generados durante el inicio de la aplicación se utilizó:

```
tail -f app.log
```

Durante el arranque se esperó encontrar un mensaje similar a:

```
Started TaskflowApiApplication
```

La aparición de este mensaje indicó que la aplicación Spring Boot había iniciado correctamente.

11. Acceso a la aplicación mediante Swagger

Con la aplicación ejecutándose en EC2, se accedió a Swagger desde un navegador web utilizando la dirección pública de la instancia:

<http://54-87-12-162:8080/swagger-ui/index.html>

El puerto **8080** corresponde al puerto utilizado por la aplicación Spring Boot.

Para permitir este acceso fue necesario que el Security Group asociado a la instancia tuviera una regla de entrada para:

TCP 8080

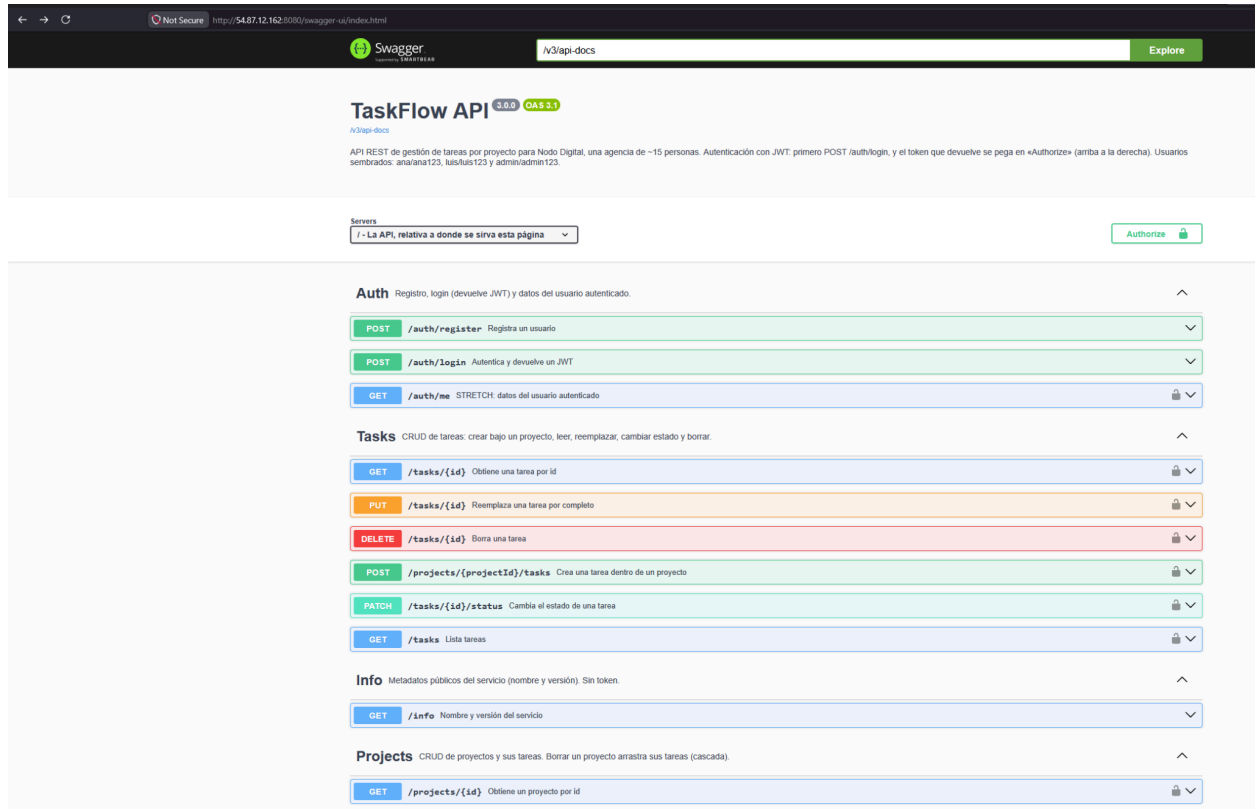
con el origen correspondiente a la configuración indicada durante la práctica.

Si el servidor responde internamente pero la aplicación no es accesible desde el navegador, una de las primeras verificaciones consiste en revisar las reglas de entrada del Security Group.

También se realizó una prueba interna desde EC2 mediante:

```
curl -s localhost:8080/swagger-ui/index.html | head -3
```

Esta prueba permite comprobar que la aplicación está respondiendo desde el propio servidor.



12. Prueba de la API

Una vez que Swagger estuvo disponible desde el navegador, se realizaron pruebas sobre los endpoints de la aplicación.

Se utilizó el usuario:

Usuario: ana

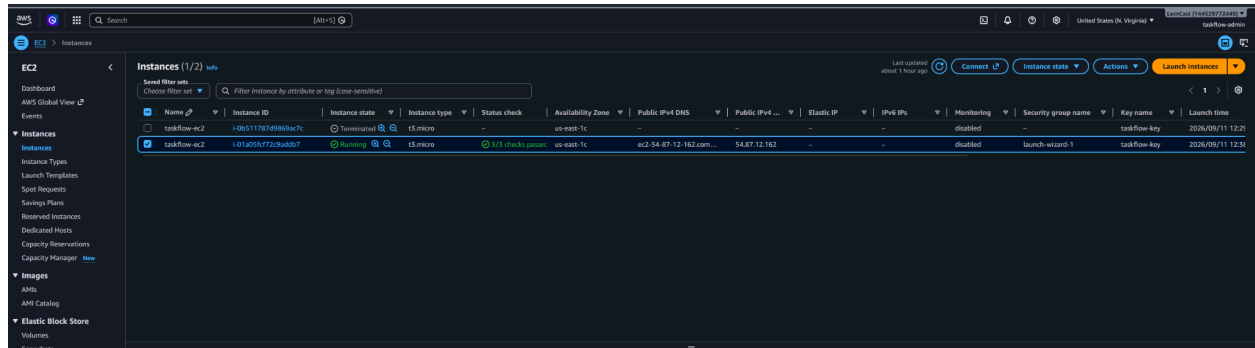
Contraseña: ana123

Después de iniciar sesión se obtuvo el token de autenticación correspondiente.

El token fue utilizado mediante la opción **Authorize** disponible en Swagger para autenticar las peticiones posteriores.

Finalmente, se utilizó uno de los endpoints de la aplicación para crear una tarea.

Esta prueba permitió comprobar no solamente que el servidor EC2 estaba accesible, sino también que la aplicación Spring Boot estaba funcionando correctamente y que sus funcionalidades principales podían utilizarse desde el exterior.

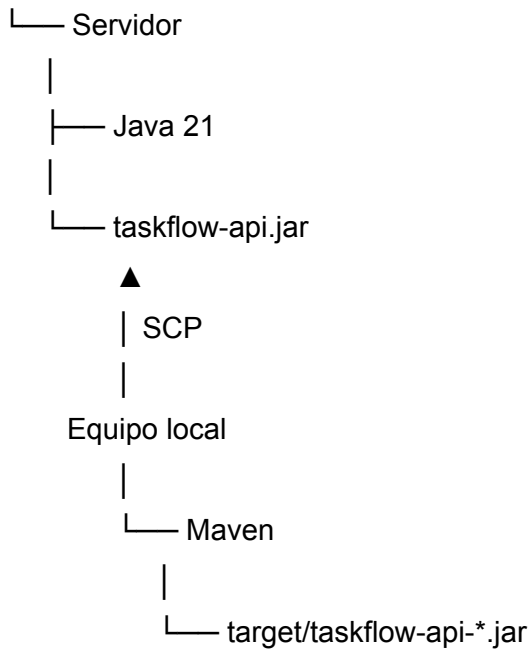


13. Flujo completo del despliegue

El proceso realizado puede resumirse de la siguiente manera:

AWS

- |
- | — IAM
 - | — Usuario y permisos
- |
- | — EC2
 - |
 - | — Instancia Amazon Linux 2023
 - | — Key Pair (.pem)
 - | — Security Group
 - | — Puerto 22 → SSH
 - | — Puerto 8080 → Aplicación
 - |



De esta manera, el proyecto pasó de un entorno local a un servidor accesible públicamente mediante la infraestructura de AWS.

14. Conclusión

La actividad permitió conocer de manera práctica el flujo básico de despliegue de una aplicación Java en AWS utilizando Amazon EC2.

Se trabajó con diferentes componentes y herramientas involucradas en un proceso real de despliegue:

- **IAM** para la administración de usuarios y permisos.
- **EC2** para proporcionar el servidor donde se ejecutaría la aplicación.
- **SSH** para establecer una conexión remota segura.
- **Git Bash** como terminal desde la cual se realizó la conexión y transferencia.
- **Java 21** como entorno necesario para ejecutar la aplicación.
- **Maven** para construir el proyecto y generar el archivo **.jar**.
- **SCP** para transferir el artefacto desde el equipo local hacia el servidor.

- **SHA-256** para verificar la integridad del archivo transferido.
- **Spring Boot** para ejecutar la aplicación.
- **Swagger** para comprobar el funcionamiento de la API desde el exterior.

Uno de los puntos importantes de la actividad fue comprender que el proceso de despliegue no necesariamente implica construir la aplicación directamente en el servidor. En este caso, el proyecto fue compilado en el equipo local debido a los recursos limitados de la instancia EC2 y posteriormente se transfirió únicamente el artefacto necesario para ejecutar la aplicación.

Finalmente, el acceso mediante Swagger y la creación de una tarea permitieron comprobar que la aplicación se encontraba funcionando correctamente en el servidor y podía ser consumida de manera remota.

Como parte de la limpieza posterior al ejercicio, la instancia EC2 puede ser terminada una vez que se hayan obtenido todas las evidencias necesarias, verificando además que no queden recursos que puedan generar cargos adicionales.